

Thomas Wilk

Assignment 2

Analysis of Algorithms for the Constraint Satisfaction Problem

This analysis covers 4 algorithms solving Constraint Satisfaction Problems, namely depth-first search with backtracking (DFSB), depth-first search with backtracking *plus* (DFSB+), min conflicts (MC), and min conflicts with random restarts (MCRR). We will explain the intuition behind each algorithm, how it performs on sample problems, and comparisons between.

In our context, a Constraint Satisfaction Problem consists of 3 components. The number of variables, the domain of each variable, and the constraints between variables. All of the constraints considered in our problems are in the form $A \neq B$. Meaning variable A and variable B cannot share the same value. This is often applied to the graph coloring problem, hence we will refer to values in the domain as colors, and variables and states.

The most basic of the four algorithms is DFSB. We iterate through every value, in every variable, and check if it satisfies the constraints given. Although this algorithm is complete (meaning it will find a solution if it exists), it is very slow. For each state s , it must check every color in its domain c . In the worst case it will check every single option, causing a time complexity of $O(c^s)$. This algorithm is the equivalent of trying to guess a password with brute force.

DFSB+ implements two heuristics to aid our search, the most restricted variable (MRV), and least restricted value (LRV). MRV is computed at each step by finding the variable which has the smallest domain remaining (excluding set variables). LRV is the value in a variable's domain which would rule out the least amount of options in the future. Both of these heuristics allow us to reduce collisions in the future, giving us a smarter search. Once we assign a value,

we check to see if removing this value from constrained domains causes any variable to have an empty domain (e.g. check if setting A leaves B with no colors). This forward checking allows us to stop iterating preemptively, saving more time. These heuristics allow us to search smarter and faster, finding a solution, if it exists.

If you like more variance in your life, this next algorithm may not be a complete bore. The Min conflicts algorithm selects random assignments to each variable, hoping for the best. Unless you have RNJesus on your side, this assignment will have some inconsistencies. It then finds the set of conflicting variables, and picks one at random. For this variable, it assigns it to the color from the domain that leaves the least amount of conflicts. It iterates until told to stop, or constraints are satisfied. This algorithm is more fun, yes, but not complete! This is a local search algorithm and may get stuck in local optima.

How do we fix this? Well the best part of the algorithm was the random start, so lets just do that again! MCRR implements random restarts to the Min conflicts algorithm. Let min conflicts run for a random amount of steps, and if it does not return anything, then just restart. This algorithm is complete, if you run it forever. But what is *a random amount of steps*? I don't know! It would not be smart to restart after 10 steps for a problem with 100 constraints. Nor would it be smart to restart after 100 million steps for a problem with 5 constraints. My attempt at solving this optimization was setting a total amount of steps to be num_constraints^3 . For a random amount of tries (50-100), split these steps evenly. Setting these parameters are definitely an art, and probably has the most implication on how well the algorithm performs.

Now that you have a general idea of how these algorithms work, let's see their performance on some problems!

States	Constraints	Colors	DFSB+		MCRR	
			States Explored	Time(Sec.)	States Explored	Average Time
20	100	4	15	.001	50	.01
50	625	4	22	.0065	1242	3.52
100	2500	4	41	.244	52421	8.85
200	10000	4	142	.134	19834	34.278
400	40000	4	147	.613	78576	124.901

From the table we can see the dominance of DFBS+ over MCRR. These are difficult problems which would take hours if not days for a human to do by hand, all done on the order of seconds by our algorithm. DFBS+ is a systematic algorithm which has very strong heuristics. This allows for it to search through the huge sample space in such a short time. MCRR is a local search algorithm. Although it is complete with time, it relies on a good starting point to escape local minima. With hundreds to thousands of constraints, it is very easy to get stuck in a local minima, causing this algorithm to feel the pain.